Student: Paul Runov

Course: ECE 3000

Subject: Handwritten Text Recognition System: Model Development and Image Processing

Date: December 14, 2024

GitHub Repository: github.com/Barashka02/handwritten-text-cnn

1 Introduction

The purpose of this project is to gain experience in building neural networks and applying image processing techniques to detect handwritten sentences. First, a Convolutional Neural Network (CNN) will be trained on the EMNIST Letters dataset to recognize hand-drawn images. Then, an image of a sentence will be segmented into individual letters, which will be fed through the model to identify the sentence.

This project consists of three main parts:

- Creation of the CNN model (with the help of ChatGPT)
- Training of the model (with the help of ChatGPT)
- Image processing and character detection

2 Creating the Model

The core of this project is a Convolutional Neural Network (CNN) designed to recognize handwritten letters. The CNN processes images by extracting important features and making predictions.

The CNN includes the following components:

- **Convolutional Layers**: Apply filters to detect patterns like edges and shapes.
- **Activation Function**: Uses ReLU to introduce non-linearity, enabling the model to learn complex patterns.
- **Pooling Layers**: Max Pooling reduces image size by retaining the most important features, speeding up training and preventing overfitting.
- **Fully Connected Layers**: Connect extracted features to make final predictions.
- **Output**: Utilizes Log-Softmax to convert predictions into probabilities, selecting the letter with the highest probability.

3 Training the Model

Training the CNN involves setting up the optimizer, loss function, and other hyperparameters, followed by running the training loop over multiple epochs.

- **Optimizer**: Adam optimizer with a learning rate of 0.001, effective for complex problems.
- **Loss Function**: Negative Log-Likelihood (NLL) loss works well with Log-Softmax for measuring prediction accuracy.

- **Hyperparameters**: Trained for 10 epochs with a batch size of 64 for training and 1000 for testing, using a momentum of 0.9.

During each epoch, the training loop performs the following steps:

1. **Forward Pass**: Input data is passed through the network to generate predictions.
2. **Loss Computation**: Loss is calculated by comparing predictions to actual labels.
3. **Backward Pass**: Gradients are computed to adjust the model's weights.
4. **Parameter Update**: Adam optimizer updates the model's weights based on gradients.

After each epoch, the model is evaluated on the test dataset by measuring:

- **Average Loss**: Indicates prediction accuracy.
- **Accuracy**: Percentage of correctly classified letters.

Consistent improvement in these metrics indicates successful training. Once training is complete, the model's state is saved for future use without retraining.

4 Image Processing and Character Detection

This process involves the preparation and analysis of images for accurate character recognition.

- **Loading and Preprocessing**: The images are loaded and converted to greyscale. Then, Otsu's method is used to binarize the image, separating characters from the background.
- **Morphological Operations**: The image is opened with a disk-shaped structuring element to disconnect letters that are touching each other.
- **Connected Components Labeling**: Letters are labeled using `scipy.ndimage.label`, with small components discarded to eliminate noise.
- **Character Segmentation**: Detected characters are analyzed and segmented based on their aspect ratios to isolate individual letters.
- **Character Prediction**: Segmented characters are padded, resized to 28×28 pixels, normalized, and passed through the trained CNN to predict the corresponding letter.
- **Space Detection**: Horizontal gaps between characters are measured. Gaps exceeding a dynamic threshold based on the median are marked as spaces, ensuring proper word separation.

4.1 Example Images

To illustrate how image segmentation works, consider the following examples:

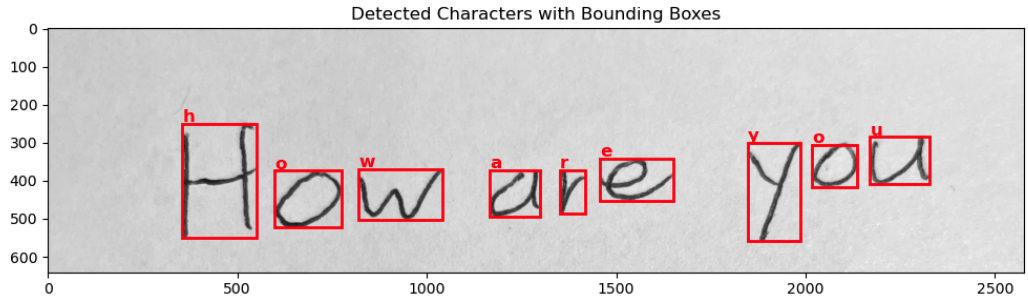

Figure 1: Example of image segmentation showing bounding boxes around detected characters.

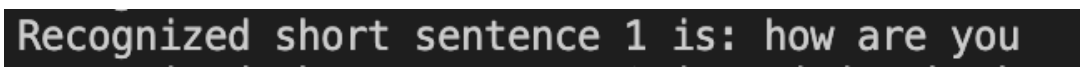

Figure 2: Final output with detected letters and reconstructed sentence.

The first image (Figure 1) demonstrates how the system identifies and segments individual characters from a handwritten sentence. Bounding boxes highlight each detected letter, ensuring accurate classification. The second image (Figure 2) shows the final recognized sentence, where spaces are correctly inserted based on the measured gaps between characters.

4.2 Image Library

The code utilizes a library of images to verify the functionality of the character detection system. The library includes:

- 10 images of numbers (one, two, etc.)
- 2 images of short sentences
- 2 images of medium-length sentences
- 2 images of long sentences

When the `process.py` file is run, it displays the images with bounding boxes around detected letters. The bounding boxes also show the detected letter. Additionally, the terminal outputs the converted sentence string for each image.

5 Results and Discussion

5.1 Performance Metrics

The CNN achieves an accuracy of around 93.5% on the test dataset, indicating its proficiency in character classification. Additionally, there is a consistent decrease in loss values across training epochs, underscoring the model's effective learning and convergence.

5.2 Challenges Encountered

Several challenges were encountered during the project:

- **Characters Touching**: Sometimes letters would be connected together or written extremely close to each other, causing multiple letters to be classified as one.
- **Space Detection**: Quantifying the distance between words is challenging due to each writer's unique writing style, which includes variable spacing between words.
- **Letter Detection**: Occasionally, letters passed through the CNN were incorrectly identified.
- **Letter 'i' Detection**: Initially, the dot from the letter 'i' was being identified as separate letters, such as 'c' and 'o'.

5.3 Solutions and Improvements

To address these challenges, several improvements were implemented:

- **Morphological Refinement**: Applying opening with a disk-shaped structuring element effectively removes small isthmuses that join letters together.
- **Median-Based Thresholding**: By calculating the median of the spaces in each sentence, spaces between words can be dynamically identified. If a space is greater than the median, it is marked as a space in the sentence.
- **Larger Padding Area**: Through experimentation, it was determined that a padding area of 25-30 pixels around the edges of the character provided accurate results when passed through the CNN.
- **Dot Removed on 'i'**: After testing, it was realized that the CNN could accurately identify an 'i' without the dot. This allowed for filtering out the small dots from detected components, simplifying the segmentation process.

6 Conclusion

This project provided valuable experience in building Convolutional Neural Networks (CNNs) and applying image processing techniques to a practical application of handwritten text recognition. Training a CNN on the EMNIST Letters dataset and developing an image segmentation pipeline revealed the complexities of handling non-uniform data.

One of the most significant challenges was accurately segmenting individual letters from connected or closely spaced characters. While the CNN effectively classified isolated letters with high accuracy, refining the segmentation algorithm was crucial for improving overall system performance. Although a basic segmentation approach was implemented due to time constraints, further development could enhance the system's robustness.

Overall, this project served as a solid foundation for understanding the interplay between neural network training and image processing. It underscored the importance of precise data preprocessing and the impact of segmentation on model accuracy. Moving forward, integrating more advanced segmentation techniques and expanding the training dataset can lead to even more reliable and efficient handwritten text recognition systems.

7 Future Work

- **Character Segmentation**: Character segmentation is the area with the greatest area for improvement. The current implementation is pretty basic. It struggles if letters are not fully 'closed' and are touching. Morphological processing either connects poorly drawn letters or disconnects letters that were touching. It wasn't possible to do both. The improvement here would be to check which letters need to be closed and which need to be opened to consistently get readable characters.
- **Auto Adjustment**: The current algorithm assumes the image contains only letters with a white background. The next improvement would be to handle any image by cropping it, straightening it, and extracting letters from there. This would also include recalculating the minimum size of objects to discard for each different image.
- **Multiple Sentences**: Currently, this algorithm only supports one sentence per image. The next step would be to add punctuation detection and the ability to process multiple sentences.
- **Upload Feature**: A Streamlit GUI was added to allow custom file uploads. Unfortunately, the image segmentation isn't consistent enough to correctly segment letters from every writing style, making this GUI not very useful. Once the image segmentation is improved to handle diverse writing styles effectively, this GUI can become a valuable tool for users to upload and process their handwritten texts.